

ML Challenge 2026: Business Entity Resolution Solution

Team Name: Quantum · **Team Members:** Aditya Singh (leader), Achita Parmar, Akshita Singh, Anuj Gupta · **Submission Date:** 27 September 2026

1. Executive Summary

We built a recall-guarded cascade:

1. Text normalisation with a transliteration map learned from the training pairs.
2. Ten complementary sparse TF-IDF blocking passes, including one reverse pass.
3. Two LightGBM rankers that shrink ~100 candidates per Source 1 record (S1) to ~10.
4. A 101-feature LightGBM matcher with a decision layer tuned for macro F0.5.

On the full labelled data the candidate recall ceiling (candidate-oracle F0.5) is **0.9956**. Out-of-fold, the matcher reaches **0.9736**. Its test-side feature pass needed ~6 more hours than we had on an 8 GB laptop. The **submitted file therefore comes from a lighter key-candidate matcher**: 13 exact normalised keys generate candidates, and a 43-feature LightGBM with the same decision layer scores them. It reaches **0.934** out-of-fold on the training data and **0.920** on the public leaderboard.

2. Methodology

2.1 Problem Analysis

- **Scale:** 2.21 M S1, 5.03 M S2 and 5.29 M S3 training records, with 7.64 M true pairs. 5.6 % of S1 are singletons, and a matched S1 has 3.5 matches on average. The test set has 1.73 M S1, including the unseen country France.
- **Structure:** every S2/S3 record matches **at most one** S1, and country labels always agree on true pairs. We therefore block within country, treating country as an open set, and enforce a one-owner assignment.
- **Noise:**
 - Names: abbreviations and legal suffixes; | -separated and d/b/a alternate names; website-style joined names.
 - Scripts: Indic native script (Devanagari and others) in India records.
 - Addresses: empty addresses, landmark addresses ("Near SBI ATM"), PO boxes, state abbreviations, typos in house numbers.
- **The hardest case is chains:** many branches share a name and differ only in address.

2.2 Solution Strategy

Approach type: blocking, then cascaded rankers, then a classifier, then a constrained decision (hybrid).

Core innovation: every pruning stage is accepted only if it loses ≤ 0.0005 candidate-oracle F0.5. A monotone-constrained pre-ranker uses MinHash record signatures. The decision layer is tuned directly on macro F0.5, singletons included.

Normalisation:

- Case and accent folding.
- Indic transliteration via Unicode character names, plus a native→Latin token dictionary learned **only from training pairs** (no external data).
- Name variants: a legal-suffix-free core name, a joined (spaceless) name and an alternate name.
- Address parsing into house number, street keys, city, state code, postal code and landmark. PO boxes are removed.

3. Candidate Generation (Blocking)

Blocking keys: ten hashed TF-IDF sparse top-k joins per country ($k = 100$). IDF is computed on $S2 \cup S3$, and very frequent features are purged ($\max_df$):

Pass	Key	Pass	Key
A	name tokens and bigrams	X	3-letter name-prefix pairs
B	house number × street keys	N	whole name × address word
D	name × address-word conjunctions	G	exact order-free address
E	address-word pairs	W	joined-name prefixes, for websites
P	phonetic consonant skeletons	RA	reverse: address-less S2/S3 records query an S1 name index ($k = 30$)

Candidate pairs: the union of all passes is cut in two steps:

1. The Stage 1.5 pre-ranker keeps the top M candidates. M is 20 for US and 40 for India; unseen countries use the maximum.
2. The Stage 2 cheap-feature ranker keeps at most 15 candidates, dropping any with p below $\tau = 0.001$.

The result is **21.8 M pairs for all 2.21 M training S1**: 9.9 per S1, 95th percentile 15. Pair recall is 0.9856 and candidate-oracle F0.5 is 0.9956.

Protecting true matches:

- Each M, budget and τ was the tightest setting within 0.0005 oracle loss of the raw union.
- Pass RA reaches address-less records that forward top-k loses among hundreds of ties.
- Monotone constraints stop the pre-ranker from discarding identical pairs (without them, identical pairs got $p \approx 1e-9$).
- Char-3-gram blocking was profiled and dropped, since it was slow and low-yield.

4. Matching Model

Features (101). No IDs, no raw strings and no country one-hot, so the model works unchanged for France.

- **Name features:**
 - String similarity: token-set, token-sort and partial ratios; Jaro-Winkler; joined-name prefix length.
 - Token weighting: IDF-weighted overlap and Jaccard; rare shared tokens; initials.
 - Variants: phonetic-skeleton similarity; best alternate-name match.

- **Address features:**
 - String similarity: token-set, token-sort and partial ratios; Jaro-Winkler; IDF overlap.
 - Numbers: house-number match, conflict and Levenshtein distance; number-set Jaccard and conflict; street-key conflict.
 - Components: postal match, conflict and 3-digit prefix; city and state match, conflict and missing flags; landmark similarity.
- **Other features:**
 - All ten pass scores and the pass count; Stage 1.5 and Stage 2 probabilities.
 - MinHash signature agreements on name, trigrams, skeleton, address and numbers.
 - Rival/context features: rank, gap to the best candidate, and candidate count per S1 and per right record.
 - Source flags: S2 vs S3, web name, native script and empty address.

Model type: LightGBM binary classifier (MIT licence), about 2,100 trees. It is cross-fitted with GroupKFold by S1, with early stopping inside each training fold. The final models are retrained on **all** labelled data; cross-validation is used only for evaluation.

Threshold selection:

1. Each S2/S3 record may be assigned only to its best-scoring S1.
2. An S1 is opened if its best candidate has $p \geq t_{\text{open}}$. Further candidates are added if $p \geq t_{\text{add}}$.
3. t_{open} and t_{add} were grid-searched on out-of-fold predictions to maximise macro F0.5, singletons included. The chosen values are US 0.63 / 0.70 and India 0.61 / 0.72; unseen countries use the global values 0.61 / 0.70.

We also tried an expected-F0.5 decision, a stacker and transitivity features; none helped.

5. Results & Error Analysis

Macro F0.5	US	India	All
Candidate-oracle, all 2.21 M training S1	0.9963	0.9945	0.9956
Full pipeline, out-of-fold (30 % S1 sample, 662 k; full-data candidates)	0.9763	0.9696	0.9736
— pair precision / recall	0.993 / 0.950	0.987 / 0.942	0.990 / 0.947
— singletons correctly left empty	0.970	0.942	0.959
Submitted key matcher , out-of-fold, all 2.21 M training S1	0.957	0.899	0.934
— public leaderboard (with France)			0.920

A 5 % development run scored 0.9865. On that run, ranking is nearly perfect: accepting exactly the top-#true candidates would score 0.995.

Most of the remaining loss is in deciding how many candidates to accept.

The error categories below overlap.

- **False positives (wrong merges, 20.8 k out-of-fold):** in 68 % the pair shares a core name, i.e. a sibling branch of the same chain. 45 % carry conflicting house numbers, and 25 % involve a right record with no address.
- **False negatives (missed matches, 88.0 k out-of-fold):** in 47 % the right record has no address. In 63 % a same-name sibling outranks the true branch. In 40 % the house number differs through typos or formatting.

6. Conclusion

Careful normalisation plus many cheap, complementary blocking passes gave a 0.9956 recall ceiling. Recall-guarded pruning kept that ceiling at ~10 candidates per S1, and the matcher reaches 0.974 out-of-fold. The main lesson: on an 8 GB machine, test-time inference must be budgeted as early as training. With about 6 more hours of compute, the same code (`ER_MODE=FINAL`) would produce the ~0.97 submission instead of the 0.92 key matcher.

Appendix

A. Code Artefacts

The code is in `code/business_entity_resolution/`: source in `src/`, with a `README.md` and a pinned `requirements.txt`. Python 3.13; numpy, pandas, pyarrow, scipy, scikit-learn, LightGBM and rapidfuzz.

Submitted files. Run from `src/`:

1. `python run_pipeline.py prepare` normalises the data.
2. `python key_matcher.py train` gives the 2-fold out-of-fold score (0.934) and thresholds, and saves the fold models.
3. `python key_matcher.py test` writes `output_km/matching_results.tsv` and `candidate_pairs.tsv`, which were copied to `output/`.

How the key matcher works:

- **Candidates:** 13 exact within-country keys, among them name + street, name + house, name + city/state, alternate name, street + city, full address, and 6-character name prefix/suffix + house/city/postal. A key links a right record to every S1 carrying it, up to 3–5 S1. This gives 20.3 M training pairs (9.2 per S1), pair recall 0.884, candidate-oracle F0.5 0.956; on test, 17.9 M pairs.
- **Features (43):** key-hit flags and ambiguity; token-set, ratio and partial similarities of names and addresses, with gaps to the best rival; house, city, state and postal agreement or conflict; source flags.
- **Model and decision:** LightGBM, 2-fold GroupKFold by S1; test is scored with the average of the two fold models. The decision is the same one-owner, t_{open} / t_{add} rule as the full pipeline (t_{open} 0.6; t_{add} 0.65 for US, 0.70 for India).

Full pipeline. `ER_MODE=FINAL python run_pipeline.py auto` is resumable and writes the same two files from Stage 3. `DEV_FAST`, `DEV_MEDIUM` and `VALIDATION` modes (5 %, 20 % and GroupKFold) share the same code path and never touch the test set.